CSC 466 Course Project Proposal: Measuring the Performance of Popular Kubernetes Networking Plugins

Oct 13, 2026 · Jesse Tran V01004890

This project compares three popular networking plugins for Kubernetes: Cilium, Calico and Flannel, measuring how fast they move data, how quickly they respond and how much computer power they use, including on slow or unreliable connections.

1. What is the problem, and why is it important?

Kubernetes is software that runs many small packaged programs (containers) across a group of computers. For those programs to talk to each other, every cluster needs a networking plugin, called a CNI plugin. The plugin you choose decides how fast data moves, how quickly messages are answered and how much computer power networking uses. The popular plugins are built very differently: Flannel wraps traffic in a simple tunnel, Calico uses traditional Linux packet-filtering rules (with an option to use newer technology called eBPF), and Cilium relies mostly on eBPF, small programs that run inside the core of the operating system. Operators usually pick their CNI plugin based on features rather than on measured behavior, and rarely know how the choice affects slow or lossy connections, large sets of access rules, or encryption.

Key research questions:

- How do Cilium, Calico and Flannel compare on speed, response time and computer-power cost, for programs on the same computer and on different computers?
- How does the cost of enforcing access rules (NetworkPolicy, which controls which programs may talk to which) grow as the number of rules goes from 10 to several thousand?
- How do two common methods for controlling data flow (TCP congestion control, here CUBIC and BBR) behave over each plugin when a Linux tool called `tc netem` adds artificial packet loss, delay and speed limits?
- How much extra cost does built-in encryption (WireGuard) add on the plugins that support it?

2. What has already been done, and why is it insufficient?

Several recent studies benchmark Kubernetes CNI plugins, and this project builds on them:

- Koukis et al. [1] compare five plugins (Flannel, Calico, Cilium, Kube-router and Kube-ovn) on four versions of Kubernetes, measuring processor use, memory and data speed. They find that "lightweight" versions of Kubernetes do not consistently use fewer resources, and no plugin exceeded about 1 gigabit per second between computers.
- Miziński and Przyłucki [2] compare the traditional Linux packet-filtering approach (iptables) with Cilium's eBPF approach. The traditional approach was faster when the system was idle, while eBPF used less processor power and memory under heavy load.
- Dakić et al. [3, 4] build automated test scripts and use them to measure data speed and response time for four plugins (Antrea, Flannel, Cilium and Calico) under different packet sizes and system settings on physical servers. All plugins lose substantial performance compared with a network that has no plugin [3].

From the abstracts and summaries reviewed, these works leave three gaps. They do not measure how the cost of access rules (NetworkPolicy) grows as rules multiply. They do not compare plugins on poor connections with artificial packet loss, delay and speed limits, including the CUBIC and BBR flow-control methods [6]. And their setups differ in hardware and software versions, so numbers cannot be compared between papers. This project fills those gaps by testing every plugin on one computer setup with one network layout, using the metrics and artificial-network-problem testing described in [5].

3. What is your approach, and why could it do better or differently?

The study combines measurement with a comparison of qualitative trade-offs. It runs entirely on a local test cluster built with kind (short for "Kubernetes in Docker", which simulates a group of computers inside one laptop), so every experiment can be repeated easily.

Testbed. One "control" node and two "worker" nodes, simulated on one laptop, with each plugin installed in turn. Test programs are pinned to specific workers so the "same computer" and "different computers" cases are clearly separated. The software versions used are recorded in the logbook.

Plugins under test. Flannel (a simple tunnel-based design), Calico (using either traditional packet-filtering rules or eBPF) and Cilium (eBPF-based, which also takes over Kubernetes's built-in traffic routing), each tested with and without WireGuard (a modern encryption tool) where supported.

Workloads and metrics.

Experiment	Tool	What we measure
Bulk transfer speed	iperf3 (a standard speed-test tool; TCP and UDP)	Useful data rate, resent packets, processor use on the node
Response time	netperf and fortio (request and response test tools)	Typical (median) and worst-case (99th percentile) response time, requests per second
Many access rules	iperf3 and fortio with 10 to 5,000 NetworkPolicy rules	Speed and response time as rules increase, time to apply the rules
Poor connections	tc netem to add packet loss (0 to 5%), delay (0 to 100 ms) and speed limits; CUBIC vs BBR	Useful data rate, time to finish a transfer, recovery after losses

Method. Each setup is tested at least 10 times, and results are reported as an average with a 95% confidence range (how much the result could vary by chance). Warm-up runs are discarded, and everything is scripted in Python so tests can be rerun exactly.

Why this could do better. Running everything on one controlled machine separates the effect of the networking plugin from hardware differences, and the poor-connection tests cover situations that vendors' own benchmarks skip.

Limitation. All simulated computers share one operating system core, and traffic passes through software-based virtual links rather than a real network card. Absolute speeds will therefore not match real hardware, so results are presented as comparisons between plugins and as processor cost, and the report says so.

4. Expected Deliverables and Biweekly Timeline

Period	Planned work and milestones	Key deliverables
Oct 12 - Oct 23	Build the simulated test cluster, check that the operating system supports the needed features (eBPF, BBR, WireGuard), install each plugin, and write a basic test and a baseline speed-test script	Repository, environment log, working three-plugin testbed
Oct 26 - Nov 06	Automate speed, response-time and processor-use measurements across all plugins with repeated runs	Midterm presentation slides and initial benchmark data
Nov 09 - Nov 20	Run the access-rule tests and the poor-connection tests (loss, delay, speed limits, CUBIC vs BBR); add encrypted runs (WireGuard)	Comparison tables and plots (growth with rule count, recovery after loss)
Nov 23 - Dec 04	Analyze results with confidence ranges, finish the comparison of qualitative trade-offs, and finalize the logbook, code and report	Final project report, code repository, demo deck

5. Logbook URL, Resources, and References

Logbook: https://github.com/forluup/csc466-cni-benchmark

Resources required: a Linux laptop or virtual machine with Docker and kind (16 GB RAM recommended), the Kubernetes command-line tools (kubectl and Helm), the installation files for Cilium, Calico and Flannel, the testing tools iperf3, netperf and fortio, `tc netem` (a Linux tool that adds artificial network problems), Python scripts for automation and analysis, and GitHub for the logbook and version control.

References (pages opened and checked on Oct 13, 2026, except where noted):

1. G. Koukis, S. Skaperas, I. A. Kapetanidou, L. Mamatas and V. Tsaoussidis, "Performance Evaluation of Kubernetes Networking Approaches across Constraint Edge Environments," arXiv:2401.07674, Jan 2024 (preprint). arxiv.org/abs/2401.07674
2. K. Miziński and S. Przyłucki, "The impact of using eBPF technology on the performance of networking solutions in a Kubernetes cluster," Journal of Computer Sciences Institute, vol. 35, pp. 150-158, 2025. doi:10.35784/jcsi.7110
3. V. Dakić, J. Redžepagić, M. Bašić and L. Žgrablić, "Performance and Latency Efficiency Evaluation of Kubernetes Container Network Interfaces for Built-In and Custom Tuned Profiles,"

Electronics, vol. 13, no. 19, 3972, 2024. doi:10.3390/electronics13193972 (preprint version checked; published page not opened)
4. V. Dakić, J. Redžepagić, M. Bašić and L. Žgrablić, "Methodology for Automating and Orchestrating Performance Evaluation of Kubernetes Container Network Interfaces," Computers, vol. 13, no. 11, 283, Nov 2024. doi:10.3390/computers13110283
5. T. Samizadeh, G. Koukis, R. C. Sofia and V. Tsaoussidis, "CNI Telco-Cloud Benchmarking Considerations," IETF Internet-Draft draft-samizadeh-bmwg-cni-benchmarking-02, 22 April 2026 (work in progress; expires 24 October 2026). ietf.org
6. N. Cardwell, Y. Cheng, C. S. Gunn, S. Hassas Yeganeh and V. Jacobson, "BBR: Congestion-based congestion control," ACM Queue, vol. 14, no. 5, Sep-Oct 2016. doi:10.1145/3012426.3022184 (author list supplied by you from the ACM page)